

SKILL SESSION 13&14

Session 13 :

TASK 1 : To Parse Input Redirection, Open Files, Redirect stdin, Handle Missing Files, Restore Streams, Test File Inputs

UW PICO 5.09

File: session13_input_redirect.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>

#define MAX_INPUT 1024
#define MAX_ARGS 50

void parse_command(char *command, char **args)
{
    int i = 0;

    char *token = strtok(command, " \\t\\n");

    while (token != NULL && i < MAX_ARGS - 1)
    {
        args[i++] = token;
        token = strtok(NULL, " \\t\\n");
    }

    args[i] = NULL;
}

int main()
{
    char input[MAX_INPUT];

    printf("Input Redirection Shell\\n");
    printf("Example: cat < input.txt\\n");

    while (1)
    {
        printf("input-redirect$ ");
        fflush(stdout);

        if (fgets(input, sizeof(input), stdin) == NULL)
            break;

        input[strcspn(input, "\\n")] = '\\0';

        if (strcmp(input, "exit") == 0)
            break;

        char *redirect = strchr(input, '<');

        if (redirect == NULL)
        {
            printf("Error: Use '<' for input redirection.\\n");
            continue;
        }

        *redirect = '\\0';
```

```

char command[MAX_INPUT];
char filename[MAX_INPUT];

strcpy(command, input);

strcpy(filename, redirect + 1);

char *file = strtok(filename, " \\t\\n");

if (file == NULL)
{
    printf("Error: No input file specified.\\n");
    continue;
}

char *args[MAX_ARGS];

parse_command(command, args);

if (args[0] == NULL)
{
    printf("Error: No command specified.\\n");
    continue;
}

pid_t pid = fork();

if (pid < 0)
{
    perror("fork");
    continue;
}

if (pid == 0)
{
    int fd = open(file, O_RDONLY);

    if (fd < 0)
    {
        perror("open");
    }
}

```

```

        perror("open");
        exit(EXIT_FAILURE);
    }

    if (dup2(fd, STDIN_FILENO) < 0)
    {
        perror("dup2");
        close(fd);
        exit(EXIT_FAILURE);
    }

    close(fd);

    execvp(args[0], args);

    perror("execvp");
    exit(EXIT_FAILURE);
}
else
{
    waitpid(pid, NULL, 0);
}
}

printf("Input redirection shell terminated.\n");

return 0;
}

```

```

(base) saiabburi@Charishma-MacBook-Air ~ % mkdir SkillWeek7
mkdir: SkillWeek7: File exists
(base) saiabburi@Charishma-MacBook-Air ~ % cd SkillWeek7
(base) saiabburi@Charishma-MacBook-Air SkillWeek7 % pwd
/Users/saiabburi/SkillWeek7
(base) saiabburi@Charishma-MacBook-Air SkillWeek7 % cat
(base) saiabburi@Charishma-MacBook-Air SkillWeek7 % cat < input.txt
zsh: no such file or directory: input.txt
(base) saiabburi@Charishma-MacBook-Air SkillWeek7 % nano session13_input_redirect.c

```

```

[(base) saiabburi@Charishma-MacBook-Air SkillWeek7 % gcc -Wall -Wextra -g session13_input_redirect.c -o session13_input_redirect
[(base) saiabburi@Charishma-MacBook-Air SkillWeek7 % echo "Hello from input file!" > input.txt
[quote> cat input.txt
[quote>
[(base) saiabburi@Charishma-MacBook-Air SkillWeek7 % echo 'Hello from input file!' > input.txt
[(base) saiabburi@Charishma-MacBook-Air SkillWeek7 % cat input.txt
Hello from input file!
[(base) saiabburi@Charishma-MacBook-Air SkillWeek7 % ./session13_input_redirect
Input Redirection Shell
Example: cat < input.txt
input-redirect$ cat < input.txt
Hello from input file!
input-redirect$ wc -w < input.txt
    4
input-redirect$ exit
Input redirection shell terminated.
[(base) saiabburi@Charishma-MacBook-Air SkillWeek7 % ./session13_input_redirect
Input Redirection Shell
Example: cat < input.txt
input-redirect$ cat < missing.txt
open: No such file or directory
input-redirect$ exit
Input redirection shell terminated.

```

TASK 2 : To Parse Output Redirection, Create Output Files, Redirect stdout, Handle Permissions, Verify Output Data, Test Redirection

```

[(base) saiabburi@Charishma-MacBook-Air SkillWeek7 % nano session13_output_redirect.c ]
[(base) saiabburi@Charishma-MacBook-Air SkillWeek7 % gcc -Wall -Wextra -g session13_output_redirect.c -o session13_output_redirect ]
[(base) saiabburi@Charishma-MacBook-Air SkillWeek7 % ./session13_output_redirect ]
Output Redirection Shell
Example: ls -l > output.txt
output-redirect$ pwd > location.txt
output-redirect$ exit
Output redirection shell terminated.
[(base) saiabburi@Charishma-MacBook-Air SkillWeek7 % cat location.txt ]
/Users/saiabburi/SkillWeek7
[(base) saiabburi@Charishma-MacBook-Air SkillWeek7 % cat location.txt ]
/Users/saiabburi/SkillWeek7
(base) saiabburi@Charishma-MacBook-Air SkillWeek7 %

```

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>

#define MAX_INPUT 1024
#define MAX_ARGS 64

int main(void)
{
    char input[MAX_INPUT];

    printf("Output Redirection Shell\n");
    printf("Example: ls -l > output.txt\n");

    while (1) {
        printf("output-redirect$ ");
        fflush(stdout);

        if (fgets(input, sizeof(input), stdin) == NULL)
            break;

        input[strcspn(input, "\n")] = '\0';

        if (strcmp(input, "exit") == 0)
            break;

        if (strlen(input) == 0)
            continue;

        char *args[MAX_ARGS];
        int argc = 0;
        char *token = strtok(input, " ");

        while (token != NULL && argc < MAX_ARGS - 1) {
            args[argc++] = token;
            token = strtok(NULL, " ");
        }

        args[argc] = NULL;

        int redirect_index = -1;

        for (int i = 0; i < argc; i++) {
            if (strcmp(args[i], ">") == 0) {
                redirect_index = i;
                break;
            }
        }
    }
}
```

```

    if (redirect_index == -1 || redirect_index + 1 >= argc) {
        printf("Usage: command > output.txt\n");
        continue;
    }

    char *filename = args[redirect_index + 1];
    args[redirect_index] = NULL;

    pid_t pid = fork();

    if (pid < 0) {
        perror("fork");
        continue;
    }

    if (pid == 0) {
        int fd = open(filename,
                      O_WRONLY | O_CREAT | O_TRUNC,
                      0644);

        if (fd < 0) {
            perror("open");
            exit(EXIT_FAILURE);
        }

        if (dup2(fd, STDOUT_FILENO) < 0) {
            perror("dup2");
            close(fd);
            exit(EXIT_FAILURE);
        }

        close(fd);

        execvp(args[0], args);

        perror("execvp");
        exit(EXIT_FAILURE);
    }

    waitpid(pid, NULL, 0);
}

printf("Output redirection shell terminated.\n");

return 0;
}

```

Session 14 :

TASK 1 : To Parse Append Operator, Open Files in Append Mode, Redirect Streams, Preserve Existing Data, Validate Appends, Test Scenarios

UW PICO 5.09

File: session14_append_redirect.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>
#define MAX_INPUT 1024
#define MAX_ARGS 50

void parse_command(char *command, char **args)
{
    int i = 0;

    char *token = strtok(command, " \\t\\n");

    while (token != NULL && i < MAX_ARGS - 1)
    {
        args[i++] = token;
        token = strtok(NULL, " \\t\\n");
    }

    args[i] = NULL;
}

int main()
{
    char input[MAX_INPUT];

    printf("Append Redirection Shell\\n");
    printf("Example: echo Hello >> output.txt\\n");

    while (1)
    {
        printf("append-redirect$ ");
        fflush(stdout);

        if (fgets(input, sizeof(input), stdin) == NULL)
            break;

        input[strcspn(input, "\\n")] = '\\0';

        if (strcmp(input, "exit") == 0)
            break;

        char *redirect = strstr(input, ">>");

        if (redirect == NULL)
        {
            printf("Error: Use '>>' for append redirection.\\n");
            continue;
        }
    }
}
```

```
*redirect = '\\0';

char command[MAX_INPUT];
char filename[MAX_INPUT];

strcpy(command, input);
strcpy(filename, redirect + 2);

char *file = strtok(filename, " \\t\\n");

if (file == NULL)
{
    printf("Error: No output file specified.\\n");
    continue;
}

char *args[MAX_ARGS];

parse_command(command, args);

if (args[0] == NULL)
{
    printf("Error: No command specified.\\n");
    continue;
}

pid_t pid = fork();

if (pid < 0)
{
    perror("fork");
    continue;
}
```

```
    if (pid == 0)
    {
        int fd = open(file, O_WRONLY | O_CREAT | O_APPEND, 0644);

        if (fd < 0)
        {
            perror("open");
            exit(EXIT_FAILURE);
        }

        if (dup2(fd, STDOUT_FILENO) < 0)
        {
            perror("dup2");
            close(fd);
            exit(EXIT_FAILURE);
        }

        close(fd);

        execvp(args[0], args);

        perror("execvp");
        exit(EXIT_FAILURE);
    }

    waitpid(pid, NULL, 0);
}

printf("Append redirection shell terminated.\n");

return 0;
}
```

```
((base) saiabburi@Charishma-MacBook-Air ~ % mkdir Skill7
```

```
((base) saiabburi@Charishma-MacBook-Air ~ % cd Skill7
```

```
((base) saiabburi@Charishma-MacBook-Air Skill7 % nano session14_append_redirect.c
```

```
((base) saiabburi@Charishma-MacBook-Air Skill7 % gcc -Wall -Wextra -g session14_append_redirect.c -o session14_append_redirect
(base) saiabburi@Charishma-MacBook-Air Skill7 % ./session14_append_redirect
Append Redirection Shell
Example: echo Hello >> output.txt
append-redirect$ echo First Line >> append.txt
append-redirect$ echo Second Line >> append.txt
append-redirect$ echo Third Line >> append.txt
append-redirect$ exit
Append redirection shell terminated.
(base) saiabburi@Charishma-MacBook-Air Skill7 % cat append.txt
First Line
Second Line
Third Line
(base) saiabburi@Charishma-MacBook-Air Skill7 % █
```

TASK 2 : To Parse stderr Redirection, Redirect Error Streams, Capture Error Output, Verify File Contents, Handle Failures, Test Error Cases.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>

#define MAX_INPUT 1024
#define MAX_ARGS 50

void parse_command(char *command, char **args)
{
    int i = 0;

    char *token = strtok(command, " \\t\\n");

    while (token != NULL && i < MAX_ARGS - 1)
    {
        args[i++] = token;
        token = strtok(NULL, " \\t\\n");
    }

    args[i] = NULL;
}

int main()
{
    char input[MAX_INPUT];

    printf("Error Redirection Shell\\n");
    printf("Example: ls missing.txt 2> error.txt\\n");

    while (1)
    {
        printf("error-redirect$ ");
        fflush(stdout);

        if (fgets(input, sizeof(input), stdin) == NULL)
            break;

        input[strcspn(input, "\\n")] = '\\0';

        if (strcmp(input, "exit") == 0)
            break;

        char *redirect = strstr(input, "2>");

        if (redirect == NULL)
        {
            printf("Error: Use '2>' for error redirection.\\n");
            continue;
        }
    }
}
```

```

*redirect = '\0';

char command[MAX_INPUT];
char filename[MAX_INPUT];

strcpy(command, input);
strcpy(filename, redirect + 2);

char *file = strtok(filename, " \t\n");

if (file == NULL)
{
    printf("Error: No error output file specified.\n");
    continue;
}

char *args[MAX_ARGS];

parse_command(command, args);

if (args[0] == NULL)
{
    printf("Error: No command specified.\n");
    continue;
}

pid_t pid = fork();

if (pid < 0)
{
    perror("fork");
    continue;
}

```

```
    if (pid == 0)
    {
        int fd = open(file, O_WRONLY | O_CREAT | O_TRUNC, 0644);

        if (fd < 0)
        {
            perror("open");
            exit(EXIT_FAILURE);
        }

        if (dup2(fd, STDERR_FILENO) < 0)
        {
            perror("dup2");
            close(fd);
            exit(EXIT_FAILURE);
        }

        close(fd);

        execvp(args[0], args);

        perror("execvp");
        exit(EXIT_FAILURE);
    }

    waitpid(pid, NULL, 0);
}

printf("Error redirection shell terminated.\n");

return 0;
}
```

```

[(base) saiabburi@Charishma-MacBook-Air Skill7 % nano session14_error_redirect.c
[(base) saiabburi@Charishma-MacBook-Air Skill7 % gcc -Wall -Wextra -g session14_error_redirect.c -o session14_error_redirect
[(base) saiabburi@Charishma-MacBook-Air Skill7 % ./session14_error_redirect
Error Redirection Shell
Example: ls missing.txt 2> error.txt
error-redirect$ cat missing.txt 2> error.txt
error-redirect$ exit
Error redirection shell terminated.
[(base) saiabburi@Charishma-MacBook-Air Skill7 % cat error.txt
cat: missing.txt: No such file or directory
[(base) saiabburi@Charishma-MacBook-Air Skill7 % ls -l
total 176
-rw-r--r--  1 saiabburi  staff    34 Sep 26 10:27 append.txt
-rw-r--r--  1 saiabburi  staff    44 Sep 26 10:42 error.txt
-rwxr-xr-x  1 saiabburi  staff 34742 Sep 26 10:26 session14_append_redirect
-rw-r--r--  1 saiabburi  staff  2302 Sep 26 10:26 session14_append_redirect.c
drwxr-xr-x  3 saiabburi  staff    96 Sep 26 10:26 session14_append_redirect.dSYM
-rwxr-xr-x  1 saiabburi  staff 34725 Sep 26 10:42 session14_error_redirect
-rw-r--r--  1 saiabburi  staff  2306 Sep 26 10:41 session14_error_redirect.c
drwxr-xr-x  3 saiabburi  staff    96 Sep 26 10:42 session14_error_redirect.dSYM

```

```

(base) saiabburi@Charishma-MacBook-Air Skill7 % cat input.txt
cat output.txt
cat append.txt
[cat error.txt
cat: input.txt: No such file or directory
cat: output.txt: No such file or directory
First Line
Second Line
Third Line
cat: missing.txt: No such file or directory

```